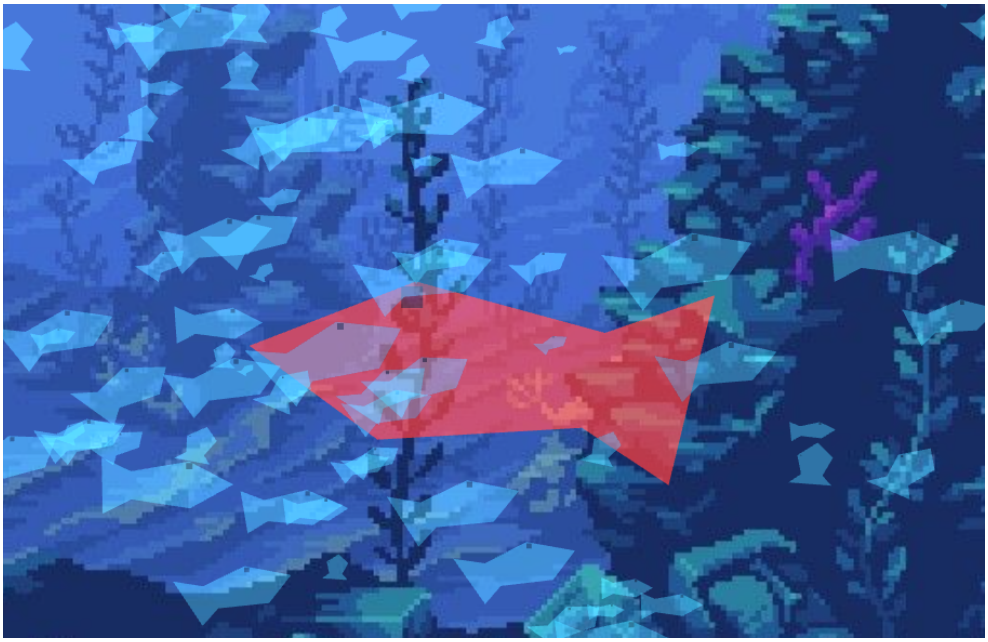
Assignment 4: Flashy Fish!

WebGL without a framework is hard. But this makes it even more rewarding when we create cool visuals!

In class, we learned how to draw multiple objects (rectangles) with different properties (colors and offset). We also made the rectangles move and some of you already added the infamous fish!! In this assignment, we will create a simple underwater world based on the things we learned.



Starter code: Please sync your fork and then use `https://cs460.org/shortcuts/04c/` as the starter code to replace `04/index.html`!

Part 1: Change createRectangle to createFish (10 Points)
In class we created the `createRectangle` method to add multiple rectangles to the `rectangles` list. Now, we want to draw a fish instead of a rectangle. Please also expand the input parameters to include `scale` and `direction` and pass them through. The following vertices look good but you can also design your own fish:

```
vertices = new Float32Array([
   0.5,  0.0, 0.0,   // 0: nose
   0.2,  0.25, 0.0,  // 1: upper body
  -0.2,  0.15, 0.0,  // 2: upper tail base
  -0.4,  0.3, 0.0,   // 3: upper tail tip
  -0.4, -0.3, 0.0,   // 4: lower tail tip
  -0.2, -0.15, 0.0,  // 5: lower tail base
   0.2, -0.25, 0.0   // 6: lower body
]);

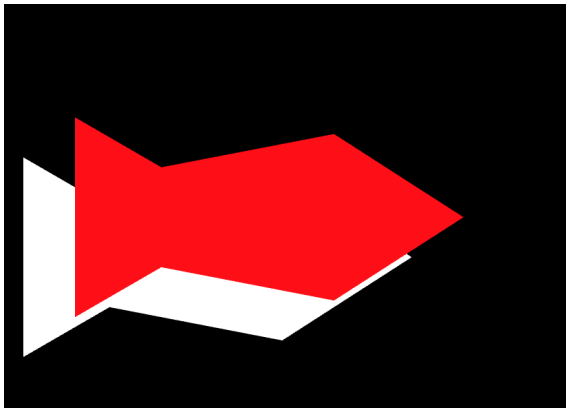
indices = new Uint8Array([
  0, 1, 6,   // main body
  1, 2, 6,   // upper mid-body
```

```
    2, 5, 6,   // rear body
    2, 3, 5,   // tail top
    3, 4, 5    // tail fin
  ]);
```

Then, please adjust your code to draw the proper number of vertices in the `animate()` function by replacing `XX` here:

```
gl.drawElements( gl.TRIANGLES, XX, gl.UNSIGNED_BYTE, 0)
```

And then, replace the `createRectangle` calls with `createFish`. For `scale` and `direction`, default values are `1`. Also, we should rename the `rectangles` list to `all_fish`. Instead of two rectangles, you should see two moving fish!



Part 2: Add the eye! (10 Points)
Every fish should have an eye! While the fish are composed of `TRIANGLES`, the eye can be rendered as `POINTS`. Since we want to have fish of different sizes with proportional eyes, we will use a dynamic `gl_PointSize`! This can be done with a new uniform in the vertex shader:

```
uniform float u_pointsize;

void main(void) {

    // ... other stuff

    gl_PointSize = u_pointsize;

}
```

Next, we will expand the `createFish` logic to define the `eye_vertex` position and then set up a new vertex buffer.

```
eye_vertex = new Float32Array([0.2, 0.2, 0.0]);

eye_v_buffer = gl.createBuffer(); // create
gl.bindBuffer( gl.ARRAY_BUFFER, eye_v_buffer ); // bind
gl.bufferData( gl.ARRAY_BUFFER, eye_vertex, gl.STATIC_DRAW ); // put data in
gl.bindBuffer( gl.ARRAY_BUFFER, null ); // unbind
```

We now need to return this new `eye_v_buffer` as well:

```
return [v_buffer, i_buffer, eye_v_buffer, color, offset, scale, direction];
```

Part 3: Render the eye! (15 Points)
We need to adjust the loop through all objects in the `animate` function to grab the new properties of the fish.

```
for( var r = 0; r < all_fish.length; r++ ) {

    // current_buffers is a list of [v_buffer, i_buffer]-pairs
    var current_buffers = all_fish[r];
    var current_v_buffer = current_buffers[0];
    var current_i_buffer = current_buffers[1];
    var current_eye_v_buffer = current_buffers[2];
    var current_color = current_buffers[3];
    var current_offset = current_buffers[4];
    var current_scale = current_buffers[5];
    var current_direction = current_buffers[6];

    // ... other stuff
```

Then, after we draw the triangles of the current fish, we need to draw the eye. First, we set the color of the eye to black with slight transparency. For this, we can re-use the `u_color` uniform on the fragment shader.

```
gl.uniform4fv( u_color, new Float32Array([0, 0, 0, .5]));
```

For the dynamic `gl_PointSize`, we will use the new `u_pointsize` uniform and adjust the size based on the current scale.

```
u_pointsize = gl.getUniformLocation( shaderprogram, 'u_pointsize' );

gl.uniform1fv( u_pointsize, new Float32Array([current_scale*20.]));
```

Finally, we need to bind the `eye_v_buffer` and issue the `gl.drawArrays` call:

```
gl.enableVertexAttribArray ( a_position );

gl.bindBuffer( gl.ARRAY_BUFFER, eye_v_buffer );

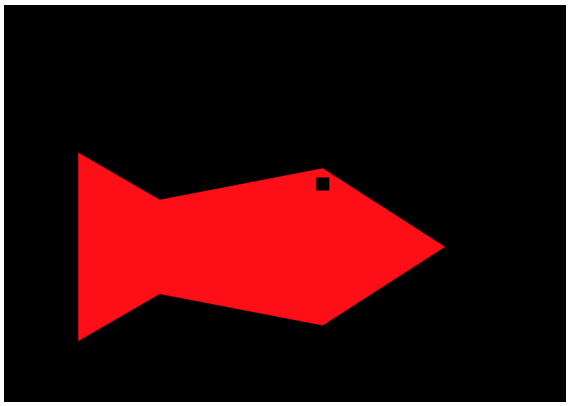
gl.vertexAttribPointer(a_position, 3, gl.FLOAT, false, 0, 0);
gl.enableVertexAttribArray(a_position);

gl.drawArrays( gl.POINTS, 0, 1);
```

At this point, your `all_fish` list should look like this:

```
all_fish = [];
all_fish.push( createFish( [0, 0, 0], [1,0,0,1], 1, 1) );
```

And a large red fish should appear!



Part 4: Add many fish (5 Points)!
Let's add some more fish! Besides the large red fish, now please create a bunch (like a 100!) of fish with random color, offset, and scale. In a loop, set them up like this:

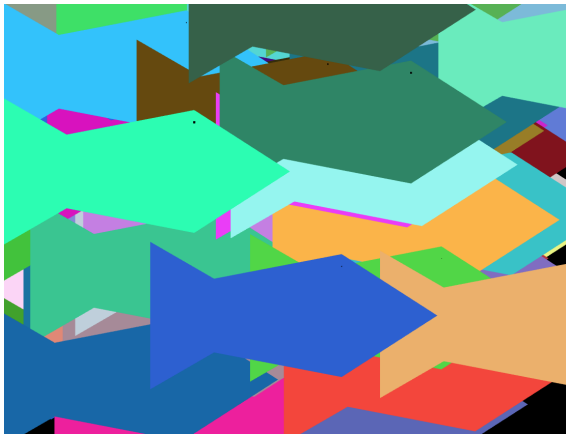
```
random_color = [Math.random(),Math.random(),Math.random(),Math.random()];

random_offset = [Math.random()-Math.random(), Math.random()-Math.random(), 0];

random_scale = Math.random()*0.3;

all_fish.push( createFish(random_color, random_offset, random_scale, 1) );
```

As always, all colors, shapes etc. can be configured to your liking!!



Note: We will do the movement next so don't worry if your fish are straight and in one line right now!

Part 5: Move the fish! (5 Points)

Let's move the fish along the x and y axes! In the `animate` loop, modify the `offset` as follows:

```
current_offset[0] += 0.01;
current_offset[1] += 0.1*Math.random();
current_offset[1] -= 0.1*Math.random();

if (current_offset[0] >= 1) {

  current_direction = -1;

}

current_offset[0] *= current_direction;
```

Hint: Remove the old offset updates (starter code lines 170-187). The new code will move the fish in x-direction and wiggle up and down in y-direction. Also, if the fish reach the right border of the viewport, they will re-appear on the left!

Part 6: Enter the matrix! (10 Points)

Instead of just modifying the `offset` vector (pure translation), we want to add a rotation of the fish by introducting a transformation matrix!

We will set up a new uniform `u_transform` in the vertex shader as follows:

```
// ... other stuff

uniform mat4 u_transform;

void main(void) {

  vec4 final_position = u_transform * vec4( a_position, 1.);
```

```
  gl_Position = final_position;

  // ... other stuff
```

The GLSL operator overloading will automatically apply the matrix multiplication! So let's now pass the translation vector (offset) as a matrix. In the `animate` loop, set up the 4x4 matrix. Note: This matrix needs to be stored in column-major ordering as follows:

```
transform = [1,0,0,0,
             0,1,0,0,
             0,0,1,0,
             current_offset[0], current_offset[1], 0, 1];
```

We don't use the `u_offset` uniform anymore (it should be removed from the shader) and instead we pass the new transformation matrix:

```
var u_offset = gl.getUniformLocation( shaderprogram, 'u_transform' );

gl.uniformMatrix4fv( u_transform, false, new Float32Array(transform));
```

Once completed, the fish should again be moving but this time controlled by a 4x4 matrix! Powerful!!

Part 7: Rotate! (10 Points)
The power of the matrix allows us to specify rotations. For a rotation around z (meaning the axis that points towards us), we can use the following matrix:

$$R_z(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 & t_x \\ \sin\theta & \cos\theta & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

where θ is an angle in radians and t_x, t_y, and t_z are our offsets.
In code (and column-major ordering), this looks like this:

```
theta = Math.random()*10 * Math.PI/180; // convert 0..10 degrees to radians

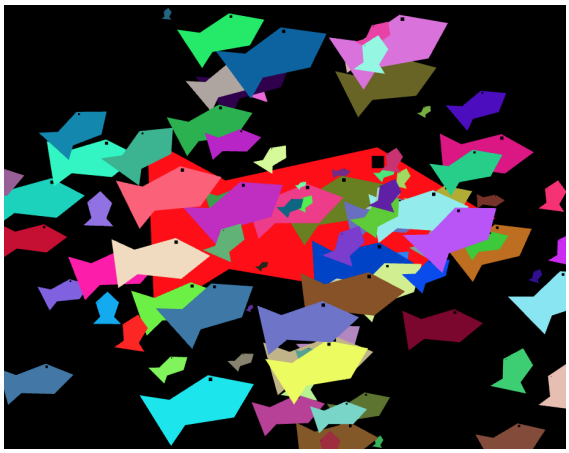
transform = [Math.cos(theta), Math.sin(theta), 0, 0,
             -Math.sin(theta), Math.cos(theta), 0, 0,
             0, 0, 1, 0,
             current_offset[0], current_offset[1], current_offset[2], 1];
```

This will not only translate using the offset, but also introduce a random 0..10 degree rotation for every fish on every draw.

Part 8: Let's incorporate scale ! (5 Points)
We want the fish to have different sizes, so let's incorporate the `current_scale` by multiplying the first three elements on the diagonal of the matrix.

```
transform = [current_scale*Math.cos(theta), Math.sin(theta), 0, 0,
             -Math.sin(theta), current_scale*Math.cos(theta), 0, 0,
             0, 0, current_scale*1, 0,
             current_offset[0], current_offset[1], current_offset[2], 1];
```

This will look cool!



Part 9: Add transparency! (5 Points)
In WebGL, we need to explicitly activate transparency even tho our colors already have an alpha channel (RGBA)! So right after calling `gl.clear` in the `animate` loop, let's put this block:

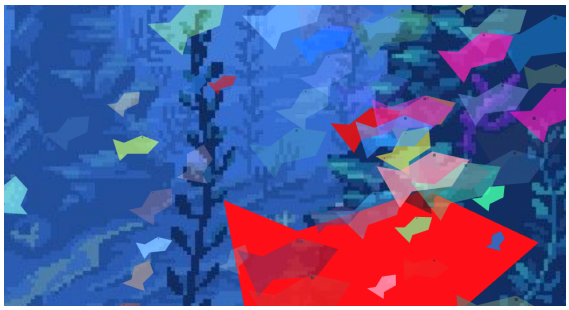
```
gl.enable(gl.BLEND);
gl.blendEquation(gl.FUNC_ADD);
gl.blendFunc(gl.SRC_ALPHA, gl.ONE_MINUS_SRC_ALPHA);

gl.disable(gl.DEPTH_TEST);
```

Part 10: Add a background! (5 Points)
We can easily add eye candy by using CSS to load a background image. Style the `body` as follows to load the official CS460 background or use your own!

```
background: url('https://cs460.org/assignments/04/bg.jpg');
background-size: cover;
```

Now this starts to look real nice!!



Part 11: Flip the big one! (10 Points)
We want the big fish to swim in the opposite direction, so let's flip it. First, we need to set the direction input to createFish for that big one to -1!

```
all_fish.push( createFish( [0, 0, 0], [1,0,0,1], 1, -1) );
```

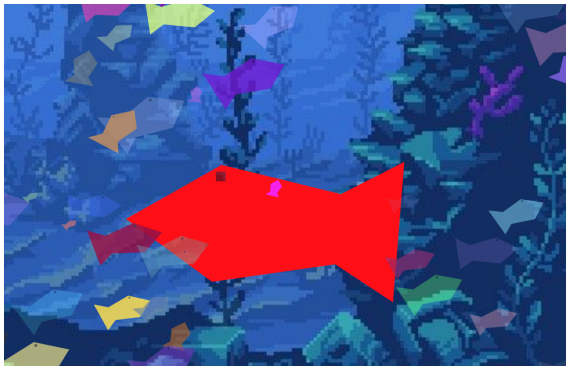
After saving and reloading, you will notice that it now hardly moves - it is fighting the current but is not flipped yet!
We need to modify the transformation matrix one more time to include the direction across the diagonal.

```
transform = [current_direction*current_scale*Math.cos(theta), Math.sin(theta), 0, 0,
             -Math.sin(theta), current_direction*current_scale*Math.cos(theta), 0, 0,
             0, 0, current_direction*current_scale*1, 0,
             current_offset[0], current_offset[1], current_offset[2], 1];
```

This will then flip the fish that have a direction of -1 (a.k.a the big one). But: the eye is now in the bottom!! To fix that, let's go back to `createFish` and change the `eye_vertex` position with a flipped y-coordinate if the direction is -1.

```
if (direction == -1) {
    eye_vertex = new Float32Array([0.2, -0.2, 0.0]);
}
```

There we go!



Part 12: Last touches! (5 Points)
We want all little fish to appear in a blue-ish pulsating color! ChatGPT to the rescue combined with the browser built-in `performance.now` object right before we pass the `current_color` to the shaders:

```
t = performance.now() * 0.001;
a = 0.6 + 0.4 * Math.sin(t * 2.0); // 0.2..1.0
current_color = new Float32Array([
    0.3 + 0.2*Math.sin(t + 0.0),
    0.6 + 0.2*Math.sin(t + 2.0),
    0.9 + 0.1*Math.sin(t + 4.0),
    a
]);

if (r == 0) {
    // the large fish aka. first object should still be red!
    current_color = [1,0,0,.7];
}

gl.uniform4fv( u_color, current_color );
```

And that's it folks!!!

Part 13: Submit the Form! (5 Points)
Finally, please push everything to your fork and double-check Github Pages! As usual, please send a pull request and then fill in the form at `https://cs460.org/assignments/04/` but don't submit it yet if you plan on doing the bonus (aka. the fun part!).

Bonus (Total 33 points):
Up to 33 extra points for the most creative solutions!! Feel free to customize any visuals or add any gameplay elements (maybe control the big fish with the keyboard or mouse, bounce of other fish, add sound...)!

SUPERBONUS (Total 133 extra points):
Port your final WebGL solution to WebGPU and submit both (please use `index_webgpu.html`)!!